

Juan Sebastián Ortiz

Senior Software Engineer — AI-Native Development Practice

Santos, SP, Brazil (UTC-3) · Remote — overlaps European and North American hours
sebasortizsanta@gmail.com · linkedin.com/in/sebastian-o · github.com/sebasortiz1989 · CV site

SUMMARY

Senior software engineer with **5 years shipping production .NET systems**, now building and delivering software through an **agent-driven development practice** I designed and operate end to end.

This is not tooling enthusiasm. It is a working system with review gates, architecture records, measured platform evidence and a real product moving through it: a Swift 6 iOS application at **127 source files across 7 modules with 100+ automated tests**, built almost entirely through AI agent sessions, with the verification discipline that makes that output trustworthy.

Depth underneath it. Five years on complex desktop UI architecture — WPF, Avalonia, .NET MAUI — including a cross-framework migration of live industrial control systems and a custom SkiaSharp rendering engine for real-time GPS on constrained hardware. Mechanical Engineer by first training, MSc Computer Science in progress. Spanish (native), Portuguese and English (professional). Comfortable in distributed, async, English-speaking teams.

PUBLIC WORK

Three repositories, all readable now — the operating model, the framework, and the application built on it.

agent-operating-model

The practice below, generalised and published. Roles as separate agent skills so the reviewer is never the author; mutation testing as the review gate; platform probes that measure behaviour instead of reasoning about it, every finding labelled **MEASURED** or **REASONED** ; written continuity through state, ADRs and an auditable board.

AvaloniaFramework —

C#, ~5,000 lines. A small MVP/navigation framework for Avalonia: a dependency-injection container with layered builders and deferred **Factory<T>** resolution, an awaitable presenter lifecycle, a navigation controller, and styled controls whose per-state appearance is set through style classes rather than looked up by resource key. MIT.

Patás & Passeios — .NET 10,

Avalonia, Dapper/SQLite. A cross-platform business application in daily use by a working dog sitter. 139 C# files, ~16,500 lines, **180 tests**, five platform heads sharing one View and Viewmodel — built on the framework above. Payment is an allocation ledger rather than a paid flag: it settles an arbitrary amount across outstanding services and banks the remainder as credit.

TECHNICAL SKILLS

AI-assisted engineering	Claude Code, multi-agent workflows, agent skill authoring, prompt and context design, output verification, mutation testing of agent-written tests
Languages	C#, Swift, TypeScript, SQL
.NET	.NET 8/9/10, WPF, Avalonia UI, .NET MAUI, XAML, ASP.NET Core, Entity Framework Core, Dapper, xUnit
Apple	Swift 6, SwiftUI, SwiftData, UIKit, Core Data, file coordination, Xcode
Architecture	MVVM, Domain-Driven Design, SOLID, Clean Architecture, ADR practice, modular package design
Graphics / perf	SkiaSharp, custom rendering, async/concurrency optimisation, GIS integration
Practice	Git, Azure DevOps, CI gates, code review, Scrum

AI-NATIVE ENGINEERING PRACTICE

Independent product build · Feb 2026 – present · solo, agent-driven

I build a real iOS product through AI agents, and I built the operating system that makes the output reliable. Both halves matter — the second is the one most teams are missing.

The product. Swift 6.3 / SwiftUI / SwiftData iOS client — 127 source files across 7 modules, 100+ tests green on both macOS and iOS destinations. A custom text layer over UIKit, a document store built on user-owned files with security-scoped bookmarks and coordinated reads and writes, and a JSON → Swift design-token generator with byte-identical regeneration checks.

The operating model. Eleven specialised agent roles authored as Claude Code skills — architect, platform developers, planner, researcher, strategist — each with defined scope, output location and git rules. Work moves through an hour-estimated board of 32 tickets with explicit dependency gates and two mandatory review stages.

Verification, which is the actual engineering problem. Agent-written tests that accompany agent-written code prove very little on their own, so reviews apply **mutation testing**: deliberately break the implementation and confirm the suite fails. Every user-visible change also takes a hand device test. Nine Architecture Decision Records capture the calls that reviews cannot re-derive.

Measured, not assumed. When a platform question can't be answered from documentation, I write throwaway code to measure it and keep it permanently as evidence — excluded from every build target, never maintained, each README recording the question, the verdict, and the exact Xcode/OS/device it held on. Findings are labelled *measured* or *reasoned*, because an agent that launders a guess into a fact is worse than no agent.

Three cases where measurement overruled a plausible answer

- **Framework API granularity.** Two platform APIs were candidates for the same job and the modern one was the obvious pick. A probe measured its output as an order of magnitude coarser than the task needed — correct in isolation, useless here. The older, lower-level API was the right route, and the finding became a written implementation rule rather than a preference.
- **Undocumented ordering behaviour.** Registering an item in a system-provided UI component put it in a different, far less visible position depending on *how* it was registered — a distinction the documentation does not draw. Ordering became an acceptance criterion, guarded by a test asserting position rather than presence.
- **A check that verified the wrong thing.** A sync script reported a directory "in sync" — truthfully, against a destination that had been formally withdrawn. A drift check cannot notice its own comparison is wrong; now a standing review question.

PROFESSIONAL EXPERIENCE

Senior Full Stack .NET Developer — VERION Oleohidráulica Brasil

Santos, SP, Brazil · Jan 2022 – Present

Engineering software for precision-agriculture operations across desktop and embedded hardware targets.

- **Cross-framework architectural migration.** Participated in migrating core agricultural control systems from WPF to **.NET MAUI and Avalonia**, unifying one codebase across multiple hardware targets while keeping business-critical systems in service.
- **High-performance custom rendering.** Built a mapping and rendering engine on **SkiaSharp** delivering real-time GPS visualisation on resource-constrained devices.
- **Concurrency and performance.** Refactored synchronous legacy code to `async / await`, resolving UI-thread blocking that had degraded responsiveness across the product suite.
- **Standards and technical debt.** Drove Domain-Driven Design and SOLID adoption, built a reusable XAML component library on strict MVVM, and ran code review and xUnit coverage.
- **Domain translation.** Worked directly with hydraulics and agronomy engineers to turn physical-system requirements into software specifications.

Unity & WPF Developer — Life Is The Game

Remote · May 2021 – Dec 2021

- Complex WPF/XAML interfaces on strict MVVM, decoupling game logic from presentation.
- Diagnosed and fixed UI memory leaks; integrated Unity Analytics for behavioural data.

Mechanical Engineer — Jotagallo S.A.

Colombia · Mar 2013 – Apr 2021

- CAD/CAM design (Autodesk Inventor, SolidWorks) and CNC programming for precision manufacturing.
- Ran materials-laboratory operations and metal analysis against engineering specifications — measurement discipline that carries directly into how I evaluate system behaviour today.

EDUCATION

- **MSc Computer Science** — Heriot-Watt University (remote) · Feb 2025 – present
- **BSc Mechanical Engineering** — EAFIT University · 2012

CERTIFICATIONS

Meta iOS Developer (Meta) · Parallel Programming with C# and .NET · ASP.NET Core + React + Redux

LANGUAGES

Spanish (native) · Portuguese (professional) · English (professional)